

基于 Calcite 的 ShardingSphere 联邦查询引擎实现

端正强
duanzhengqiang@apache.org

01

Calcite 入门介绍

02

ShardingSphere 联邦查询引擎实现

03

ShardingSphere 联邦查询引擎规划



Calcite Demo

假设 HR 手里有两个 Excel 表格，一个是部门信息，另外一个员工信息，她想要对员工和部门信息进行统计，请问有哪些解决方法？

	DEPTS.csv	EMPS.csv
1	DEPTNO:int,NAME:string	
2	10,"Sales"	
3	20,"Marketing"	
4	30,"Accounts"	

	DEPTS.csv	EMPS.csv
1	EMPNO:int,NAME:string,DEPTNO:int,GENDER:string,CITY:string,EMPID:int,AGE:int,SLACKER:boolean,MANAGER:boolean,JOINEDAT:date	
2	100,"Fred",10,,,30,25,true,false,"1996-08-03"	
3	110,"Eric",20,"M","San Francisco",3,80,,false,"2001-01-01"	
4	110,"John",40,"M","Vancouver",2,,false,true,"2002-05-03"	
5	120,"Wilma",20,"F",,1,5,,true,"2005-09-07"	
6	130,"Alice",40,"F","Vancouver",2,,false,true,"2007-01-01"	
7		



Calcite Demo

一种可行的解决办法：使用 **Calcite** 作为计算层引擎，快速连接各种类型的数据存储，提供 SQL 查询能力。

```
strongduanmu@duanzhengqiangdeMacBook-Pro ~$ cd ~/IdeaProjects/calcite/example/csv
strongduanmu@duanzhengqiangdeMacBook-Pro ~/IdeaProjects/calcite/example/csv$ ./sqlline • calcite 命令行工具
Building Apache Calcite 1.33.0-SNAPSHOT
sqlline version 1.12.0
sqlline> !connect jdbc:calcite:model=src/test/resources/model.json admin admin
Transaction isolation level TRANSACTION_REPEATABLE_READ is not supported. Default (TRANSACTION_NONE) will be used instead.
0: jdbc:calcite:model=src/test/resources/mode> !tables • 查看所有表
```

TABLE_CAT	TABLE_SCHEM	TABLE_NAME	TABLE_TYPE	REMARKS	TYPE_CAT	TYPE_SCHEM	TYPE_NAME	SELF_REFERENCING_COL_NAME	REF_GENERATION
	SALES	DEPTS	TABLE						
	SALES	EMPS	TABLE						
	SALES	SDEPTS	TABLE						
	metadata	COLUMNS	SYSTEM TABLE						
	metadata	TABLES	SYSTEM TABLE						

```
0: jdbc:calcite:model=src/test/resources/mode> SELECT * FROM emps; • 查看员工信息
```

EMPNO	NAME	DEPTNO	GENDER	CITY	EMPID	AGE	SLACKER	MANAGER	JOINEDAT
100	Fred	10			30	25	true	false	1996-08-03
110	Eric	20	M	San Francisco	3	80		false	2001-01-01
110	John	40	M	Vancouver	2	null	false	true	2002-05-03
120	Wilma	20	F		1	5		true	2005-09-07
130	Alice	40	F	Vancouver	2	null	false	true	2007-01-01

```
5 rows selected (1.132 seconds)
0: jdbc:calcite:model=src/test/resources/mode> SELECT d.name, COUNT(*) FROM emps AS e RIGHT JOIN depts AS d ON e.deptno = d.deptno GROUP BY d.name; • 统一每个部门的员工数
```

NAME	EXPR\$1
Sales	1
Accounts	1
Marketing	2

```
3 rows selected (0.255 seconds)
```



什么是 Calcite



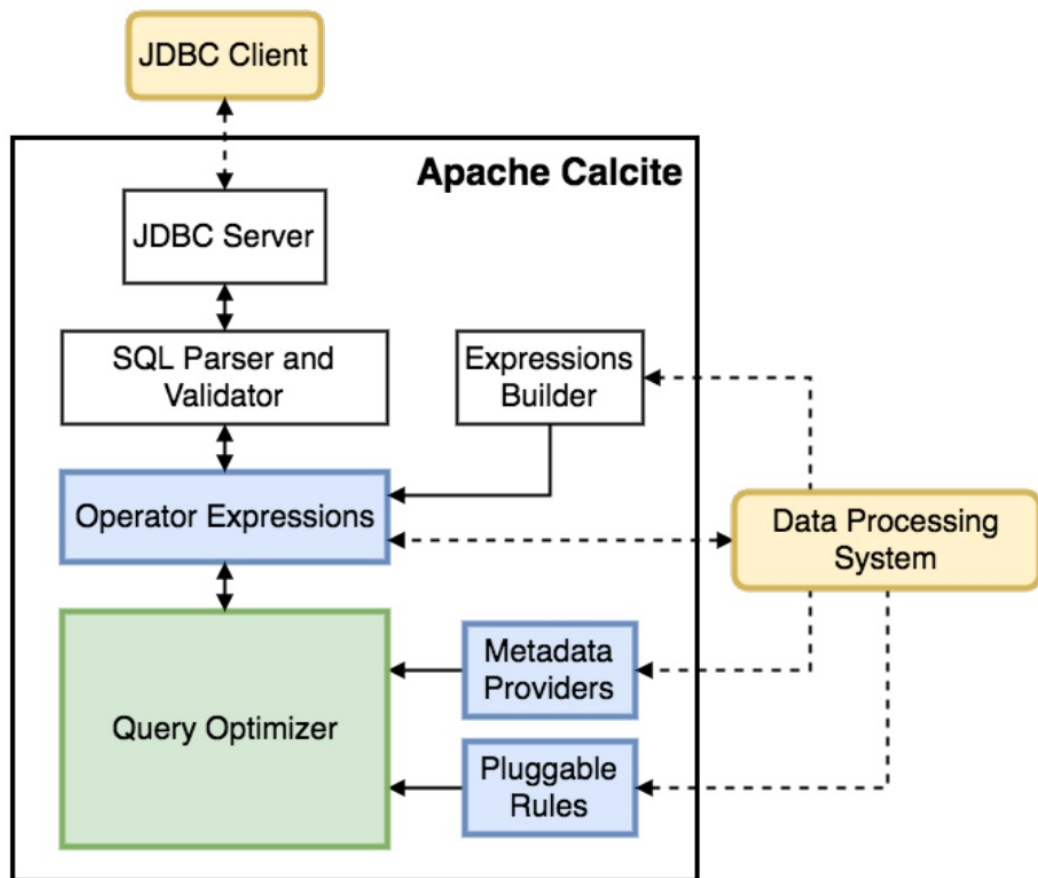
使用 Calcite 的知名项目



- ◆ Calcite 是一个动态数据管理框架，提供了 SQL 解析、SQL 校验、SQL 查询优化、SQL 生成等典型数据库管理功能；
- ◆ Calcite 设计目标是 one size fits all，为不同存储引擎提供统一的 SQL 查询引擎；
- ◆ Calcite 具有灵活性（Flexible）、组件可插拔（Embeddable）和可扩展（Extensible）3 大核心特性，Calcite 的解析器、优化器都可以作为独立的组件使用；



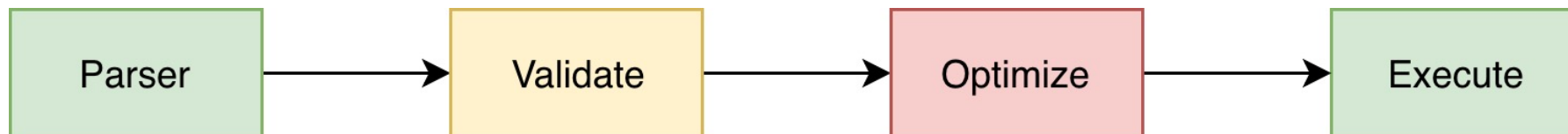
Calcite 架构



- ◆ 最外层是 JDBC Client 和数据处理系统，JDBC Client 用于连接 Calcite，支持 Local 和 Remote 两种方式，数据处理系统则用于对接不同的数据存储引擎；
- ◆ 内层是 Calcite 核心架构的流程性组件，包括负责接收 JDBC 请求的 JDBC Server，负责解析 SQL 语法的 **SQL Parser**，负责校验 SQL 语义的 **SQL Validator**，以及负责构建算子表达式的 Expression Builder；
- ◆ 算子表达式、元数据提供者、可插拔优化规则是用于适配不同逻辑的适配器，这些适配器都可以进行灵活地扩展；
- ◆ **查询优化器是整个 Calcite 的核心**，负责对逻辑执行计划进行优化，**基于 RBO 和 CBO 两种优化模型**，得到可执行的最佳执行计划。



Calcite 执行流程



- ◆ 解析 SQL：把 SQL 转换成 AST（抽象语法树），在 Calcite 中用 SqlNode 来表示；
- ◆ 语法检查：根据 Calcite 元数据信息进行语法验证，验证之后还是用 SqlNode 表示 AST 语法树；
- ◆ 语义分析：根据 SqlNode 及元数据信息构建 RelNode 树，也就是最初版本的逻辑计划（Logical Plan）；
- ◆ 逻辑计划优化：根据前面生成的逻辑计划按照相应的规则（Rule）进行优化；
- ◆ 物理执行：生成物理计划，并基于统计信息和代价模型，找到最优的物理执行计划进行执行。



Calcite 核心组件

- ◆ Catalog : 用于定义SQL 查询所需的元数据和命名空间信息 ;
- ◆ SQL parser : 解析引擎检验 SQL 语法并将其转换为抽象语法树 (AST) ;
- ◆ SQL validator : 根据 catalog 中提供的元数据校验抽象语法树 , 并进行标准化处理 ;
- ◆ Query optimizer : 将 AST 转换为逻辑计划 , 并根据规则对逻辑计划进行优化 , 并根据统计信息和代价模型生成最优物理计划 ;
- ◆ SQL executor : 根据物理计划生成可执行的代码 ;



Calcite 核心组件——Catalog

- ◆ 定义 Calcite 查询所需的元数据及命名空间。
- ◆ Schema
 - ◆ schema 和 table 的集合
 - ◆ schema 可以任意层级嵌套，可以支持 MySQL 两层元数据及 PostgreSQL 三层元数据结构；
- ◆ Table
 - ◆ 表示单个数据集合；
 - ◆ 字段用 RelDataType 定义；
 - ◆ 提供优化所需的表的统计信息；
- ◆ RelDataType
 - ◆ 代表数据集中的字段，RelDataTypeField 表示单个字段；
 - ◆ 支持所有 SQL 类型，以及 struts 和 arrays 类型；



Calcite 核心组件——Catalog

```
public interface Schema {  
  
    Table getTable(String name);  
  
    Set<String> getTableNames();  
  
    Schema getSubSchema(String name);  
  
    Set<String> getSubSchemaNames();  
  
}
```

```
public interface Table {  
  
    RelDataType getRowType(RelDataTypeFactory typeFactory);  
  
    Statistic getStatistic();  
  
    Schema.TableType getJdbcTableType();  
}  
  
public interface Statistic {  
    Double getRowCount();  
  
    boolean isKey(ImmutableBitSet columns);  
  
    List<RelCollation> getCollations();  
  
    RelDistribution getDistribution();  
}
```



Calcite 核心组件——Catalog

Catalog 对象在 SQL 中对应位置：

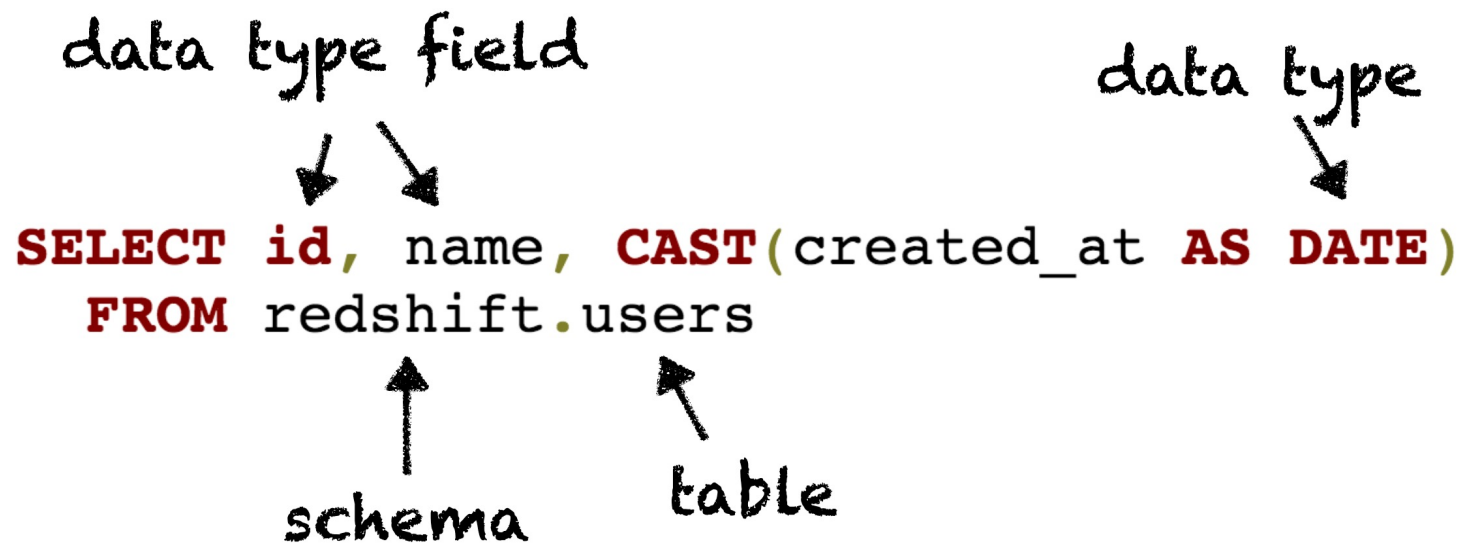
data type field data type

↓ ↓ ↓

```
SELECT id, name, CAST(created_at AS DATE)  
FROM redshift.users
```

↑ ↑

schema table

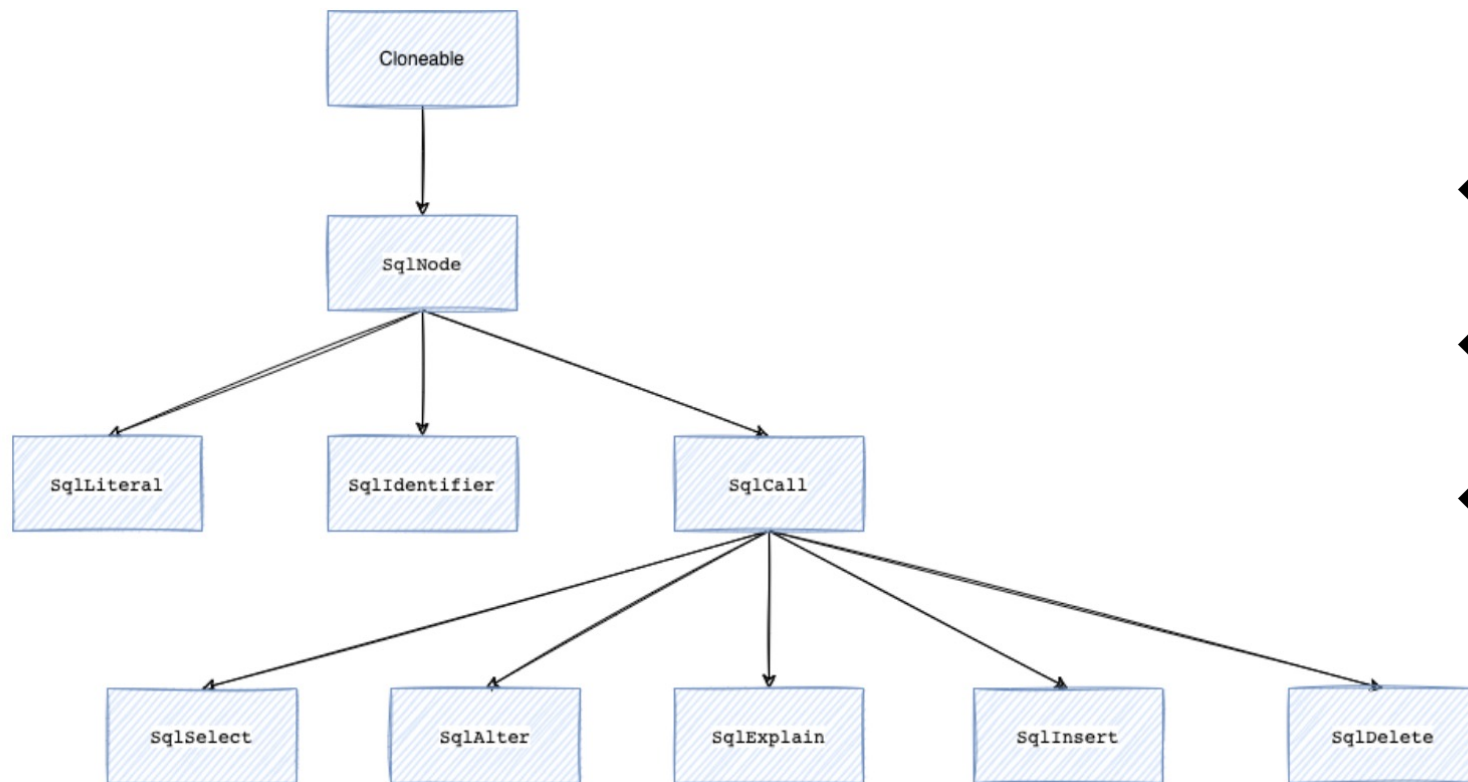


Calcite 核心组件——SQL Parser

- ◆ 使用 JavaCC 编写 SQL 解析逻辑；
- ◆ 输入的查询 SQL 被解析为抽象语法树 (AST)；
- ◆ Calcite 中使用 **SqlNode** 代表 Token ；
- ◆ 可以从 SQL 字符串转换为 SqlNode ，也可以从 SqlNode 转换为 SQL 字符串；
- ◆ Calcite SQL 解析完全理解 SQL 语义；



Calcite 核心组件——SQL Parser

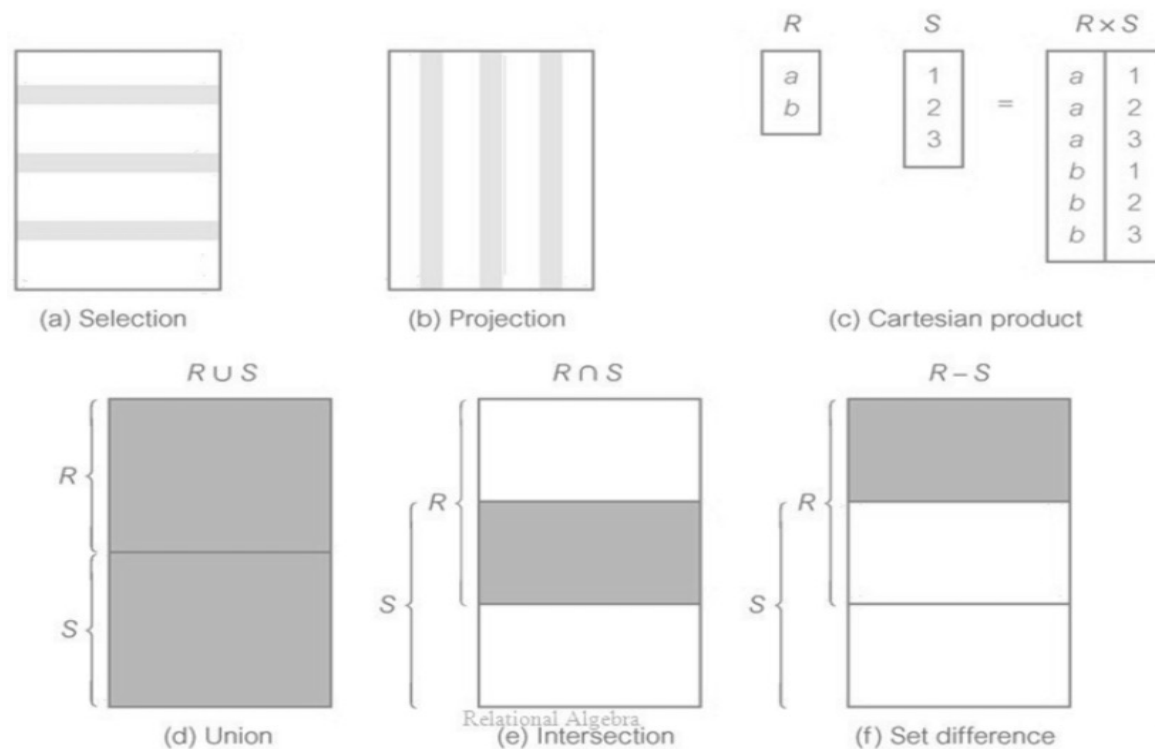


- ◆ `SqlLiteral` : 代表用户输入的常量，即字面量；
- ◆ `SqlIdentifier` : 代表用户输入的标识符，例如：表名、字段名等；
- ◆ `SqlCall` : 每一个具体的操作都可以对应一个 `SqlCall`，例如查询语句对应的 `SqlSelect`；



Calcite 核心组件——Query Optimizer

关系代数是 Calcite 查询优化的基础理论，任何一个查询都可以表示成由关系运算符组成的树，基于关系代数的等价变化原则对 SQL 进行优化。关系代数支持了如下：选择、投影、笛卡尔积、并、交、差等基本运算。



Calcite 核心组件——Query Optimizer

如下是根据关联查询 SQL 生成的逻辑计划，逻辑计划代表了执行查询的顺序。



```

LogicalProject(user_id=[0], user_name=[1], order_id=[5])
  LogicalFilter(condition=[>(0, 50)])
    LogicalJoin(condition=[=(0, 6)], joinType=[inner])
      LogicalTableScan(table=[[USERS]])
      LogicalTableScan(table=[[ORDERS]])
  
```



Calcite 核心组件——Query Optimizer

Calcite 优化器 Planner 实现了 RelOptPlanner 接口，目前内置了两种类型的优化器：**HepPlanner** 和 **VolcanoPlanner**。

- ◆ HepPlanner 是一个**启发式的优化器**，使用**预先定义的优化规则**去匹配逻辑计划，直到无匹配结果或达到最大匹配次数。
- ◆ 启发式的优化器**性能优于基于代价的优化器**；
- ◆ 如果规则对计划做出相反的更改，则**有无限递归的风险**。
- ◆ VolcanoPlanner 是一个**基于代价的优化器**，以迭代的方式去匹配规则，在每次迭代中选择成本最低的计划，cost 由关系表达式提供；
- ◆ 不是所有执行计划都能被包含在代价计算中；
- ◆ 当 **cost 不继续显著降低时，将会停止优化**。



Calcite 核心组件——Query Optimizer

HepPlanner 逻辑优化的常用方式：

- ◆ 列裁剪 (Prune unused fields) ；
- ◆ 子查询转换为关联查询 (Convert subqueries to joins) ；
- ◆ Join 重排序 (Reorder joins) ；
- ◆ 投影下推 (Push down projections) ；
- ◆ 谓词下推 (Push down filters) 。



Calcite 核心组件——Query Optimizer

投影下推和谓词下推示例：

```
SELECT u.id AS user_id, u.name AS user_name, o.id AS order_id
FROM users u INNER JOIN orders o ON u.id = o.user_id
WHERE u.id > 50
```

```
LogicalProject(user_id=[0], user_name=[1], order_id=[5])
  LogicalFilter(condition=[>($0, 50)])
    LogicalJoin(condition=[=(0, 6)], joinType=[inner])
      LogicalTableScan(table=[[USERS]])
      LogicalTableScan(table=[[ORDERS]])
```

```
LogicalProject(user_id=[0], user_name=[1], order_id=[5])
  LogicalJoin(condition=[=(0, 6)], joinType=[inner])
    LogicalProject(ID=[0], NAME=[1])
      LogicalFilter(condition=[>($0, 50)])
        LogicalTableScan(table=[[USERS]])
    LogicalProject(ID=[0], USER_ID=[1])
      LogicalTableScan(table=[[ORDERS]])
```

Diagram illustrating query optimization with annotations:

- push down project**: An arrow points from the `LogicalProject(ID=[0], NAME=[1])` node to the `LogicalJoin` node, indicating that the projection can be pushed down to the join operation.
- push down filter**: An arrow points from the `LogicalFilter(condition=[>($0, 50)])` node to the `LogicalJoin` node, indicating that the filter can be pushed down to the join operation.



Calcite 核心组件——Query Optimizer

VolcanoPlanner 物理优化的重点：

- ◆ Cost 由每一个 RelNode 提供；
- ◆ RelOptCost 对象代表了 Cost；
- ◆ Cost 通常包含了行数、IO 和 CPU 代价；
- ◆ Cost 估计是相对的；
- ◆ 统计信息可以提升 Cost 估计的准确性；



Calcite 核心组件——Query Optimizer

VolcanoPlanner 物理优化示例：

```
SELECT u.id AS user_id, u.name AS user_name, o.id AS order_id
FROM users u INNER JOIN orders o ON u.id = o.user_id
WHERE u.id > 50
```

```
SparkProject(user_id=[0], user_name=[1], order_id=[2])
  SparkJoin(condition=[(0, 3)], joinType=[inner])
    JdbcToSparkBridge
      JdbcProject(ID=[0], NAME=[1])
        JdbcFilter(condition=[>(0, 50)])
          JdbcTableScan(table=[[acme, site, PUBLIC, USERS]])
    JdbcToSparkBridge
      JdbcProject(ID=[0], USER_ID=[1])
        JdbcTableScan(table=[[acme, cart, PUBLIC, ORDERS]])
```

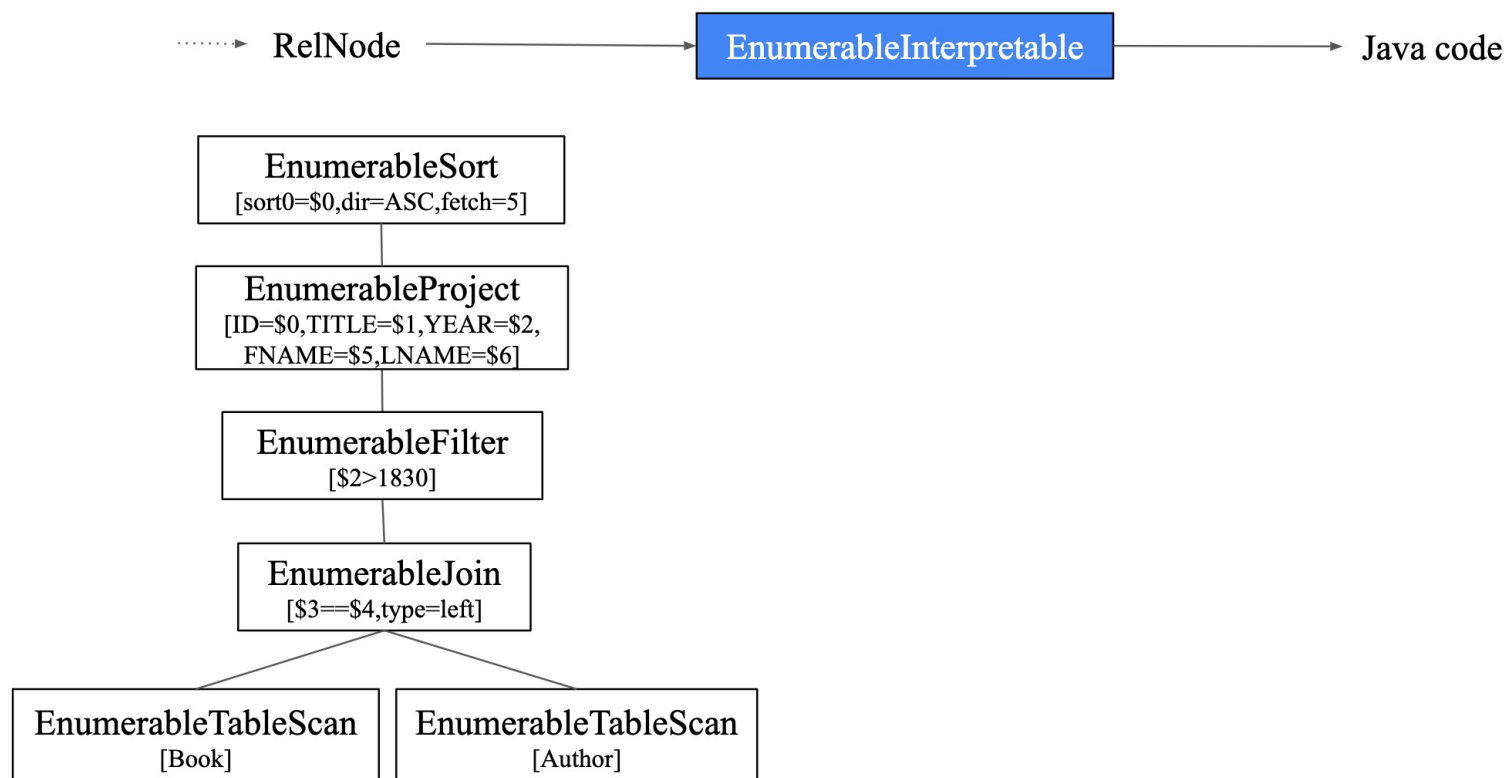
```
SparkProject(user_id=[0], user_name=[1], order_id=[2]): rowcount = 750.0, cumulative cost = {2122.0 rows, 1.0 cpu, 80.0 io}, id = 21748
  SparkJoin(condition=[(0, 3)], joinType=[inner]): rowcount = 750.0, cumulative cost = {1222.0 rows, 1.0 cpu, 80.0 io}, id = 21747
    JdbcToSparkBridge: rowcount = 50.0, cumulative cost = {161.0 rows, 81.0 cpu, 80.0 io}, id = 21748
      JdbcProject(ID=[0], NAME=[1]): rowcount = 50.0, cumulative cost = {160.0 rows, 80.0 cpu, 80.0 io}, id = 21749
        JdbcFilter(condition=[>(0, 50)]): rowcount = 50.0, cumulative cost = {120.0 rows, 80.0 cpu, 80.0 io}, id = 21750
          JdbcTableScan(table=[[acme, site, PUBLIC, USERS]]): rowcount = 100.0, cumulative cost = {80.0 rows, 80.0 cpu, 80.0 io}, id = 21751
    JdbcToSparkBridge: rowcount = 100.0, cumulative cost = {161.0 rows, 1.0 cpu, 80.0 io}, id = 21750
      JdbcProject(ID=[0], USER_ID=[1]): rowcount = 100.0, cumulative cost = {160.0 rows, 0.0 cpu, 80.0 io}, id = 21752
        JdbcTableScan(table=[[acme, cart, PUBLIC, ORDERS]]): rowcount = 100.0, cumulative cost = {80.0 rows, 0.0 cpu, 80.0 io}, id = 21753
```



Calcite 核心组件——SQL Executor

物理计划执行的方式：

- ◆ 使用内置的 Enumerable 算子，通过 EnumerableInterpretable#toBindable 生成 Java 代码执行（如下）；
- ◆ 自定义物理算子和执行代码。



01

Calcite 入门介绍

02

ShardingSphere 联邦查询引擎实现

03

ShardingSphere 联邦查询引擎规划



ShardingSphere 联邦查询引擎实现——第一版

[New feature] Federated SQL query and Query optimization are going to sail out. #8284

[Edit](#)[New issue](#)[Open](#)[24 of 33 tasks](#)

tristaZero opened this issue on Nov 22, 2020 · 50 comments · Fixed by #11079



tristaZero commented on Nov 22, 2020 · edited

Member

Hi community,

As you know, ShardingSphere has made a lot of efforts on SQL parser and provided a great independent SQL parser to help users parse SQL.

Based on this substantial work, we plan to do query optimization to optimize the input SQLs from users and produce an optimized SQL query plan to improve query efficiency. Plus, the federated SQL query feature (Like join query from different instances) is another essential highlight for our next release. :)

We will leverage Apache Calcite, an excellent framework to implement two of the features. Currently, three main work focus are presented here.

- ✓ The investigation of RBO (Rule-Based Optimization) and CBO (Cost-Based optimization) ,i.e., Hep planner engine and Volcano planner engine. (Mechanism, usage, pros and cons).
- ✓ The investigation of Calcite adaptor, especially TranslatableTable API.
- ✓ How to transform the parsed result of ShardingSphere to the algebra of Calcite
- ✓ Call some of the optimization rules to process the relational expression
- ✓ Implement Calcite adaptor to join SQLs from different instances
- ✓ Combine sharding tables with Calcite adaptors.

Actually, there are plenty of works to do on this issue. We are in the investigation phase now and will seek contributors for this issue later. If you are interested in this one, please give it a watch. 🙄

Assignees

guimingyue

strongduanmu

junwen12221

wgy8283335

tristaZero

Labels

in: kernel

type: new feature

Projects

None yet

Milestone

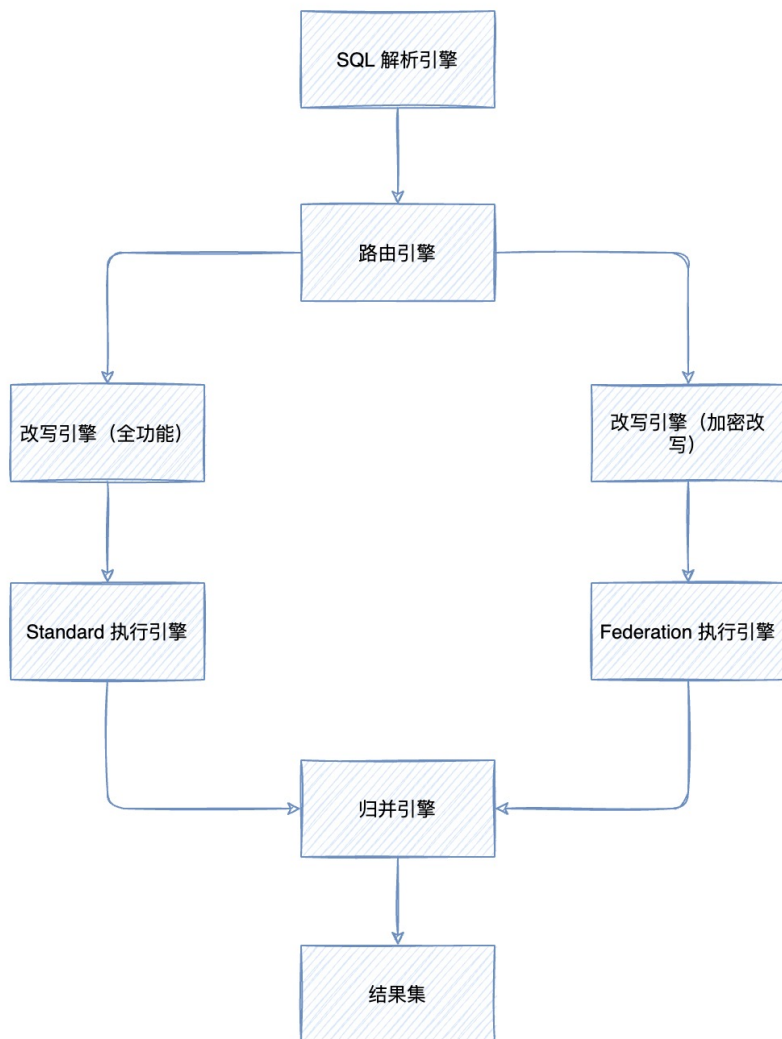
No milestone

Development

Successfully merging a pull request may close this issue.



ShardingSphere 联邦查询引擎实现——第一版



实现思路：

- ◆ 使用 Calcite JDBC 实现基础联邦查询，支持跨库查询、子查询、聚合查询等复杂 SQL；
- ◆ 使用真实元数据注册到 Calcite，Calcite 需要提前路由、改写为真实 SQL 再交由联邦查询引擎处理；
- ◆ SQL 生成器根据 ProjectableFilterableTable 接口参数生成 SQL 并执行；
- ◆ 通过归并引擎将真实结果集归并为逻辑结果集返回给用户。

存在的问题：

- ◆ 默认使用 Calcite 内置优化规则，没有集成优化模块 ShardingSphereOptimizer；
- ◆ 存在着两次解析问题（Calcite 解析和 ShardingSphere 解析）；
- ◆ 下推 SQL 生成后全路由，会导致在所有真实表中执行查询，当真实表数量非常多时，会导致查询效率降低；
- ◆ 两套元数据，维护成本高；

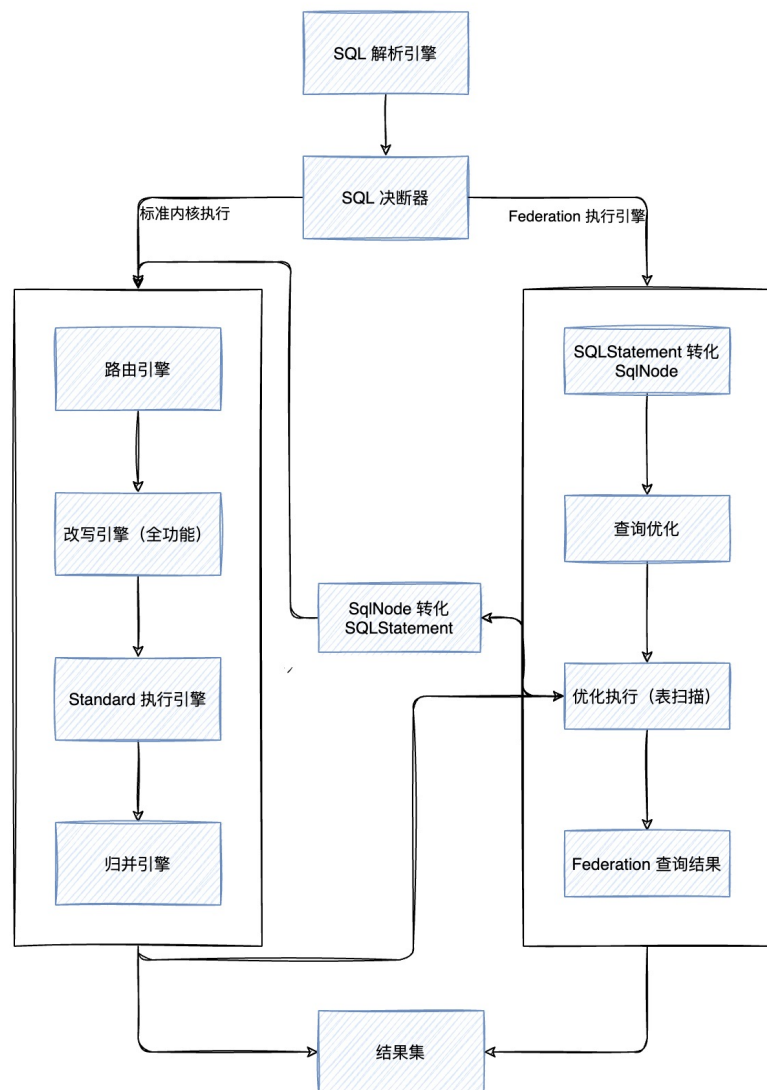
ShardingSphere 联邦查询引擎实现——第二版

实现思路：

- ◆ 参考 Calcite JDBC，自行实现联邦查询引擎执行流程：
 - ◆ ShardingSphere SQL 解析；
 - ◆ SQLStatement 转换 SqlNode；
 - ◆ SqlNode 转换 RelNode 并校验；
 - ◆ 基于优化规则进行逻辑优化；
 - ◆ 基于统计信息，代价模型进行物理优化（未实现）；
 - ◆ 物理计划执行（未实现自定义算子）；
- ◆ 统一内核逻辑元数据和真实元数据，使用逻辑元数据注册到 Calcite，逻辑 SQL 通过决断器后交由 Calcite 执行；
- ◆ 基于优化之后的物理计划生成 SQL。

存在的问题：

- ◆ 存在 SQLStatement 和 SqlNode 的相互转换，影响效率和兼容性；
- ◆ 缺乏统计信息和代价模型，暂不支持物理优化；
- ◆ 物理计划的执行依赖 Calcite 内置的 EnumerableRel 算子，性能一般且无法灵活控制；
- ◆ 缺乏内存保护机制，需要避免加载数据过多导致 OOM；



ShardingSphere 联邦查询引擎实现——第二版

联邦查询引擎改造：

- ◆ SQL Federation 由 infra 模块抽取到 kernel 模块，作为内置的功能；
- ◆ ShardingSphere 元数据和联邦查询引擎完全解耦；
- ◆ 通过 SQLFederationExecutor SPI 接口，将 Calcite JDBC 实现和自定义实现进行隔离，用户通过 sql-federation-type: ADVANCED 配置使用；

联邦查询引擎功能增强：

- ◆ 跨库关联查询，包括：INNER JOIN、CROSS JOIN、LEFT JOIN、RIGHT JOIN、FULL JOIN 和 NATURAL [XXX] JOIN；
- ◆ 子查询，包括：Projection 子查询，From 子查询和 Where 子查询；
- ◆ 支持任意组合的 MAX、MIN、SUM、AVG 和 COUNT 聚合函数；
- ◆ 支持 UNION/INTERSECT/EXCEPT/MINUS 查询；
- ◆ 支持逻辑视图查询；

```

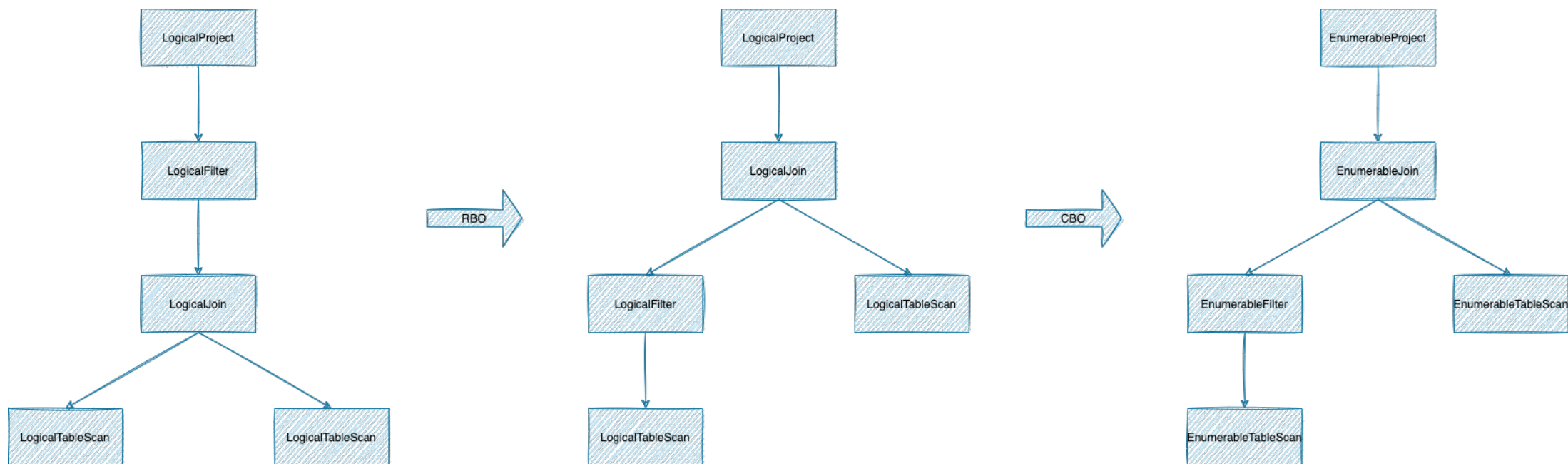
v kernel [shardingsphere-kernel]
  > authority [shardingsphere-authority]
  > data-pipeline [shardingsphere-data-pipeline]
  > parser [shardingsphere-parser]
  > schedule [shardingsphere-schedule]
  > single-table [shardingsphere-single-table]
  v sql-federation [shardingsphere-sql-federation]
    > api [shardingsphere-sql-federation-api]
    > core [shardingsphere-sql-federation-core]
    v executor [shardingsphere-sql-federation-executor]
      > advanced [shardingsphere-sql-federation-executor-advanced]
      > core [shardingsphere-sql-federation-executor-core]
      > original [shardingsphere-sql-federation-executor-original]
      > target
        ✓ pom.xml
    > optimizer [shardingsphere-sql-federation-optimizer]
      > target
        ✓ pom.xml
    > sql-translator [shardingsphere-sql-translator]
      > target
    > traffic [shardingsphere-traffic]
    > transaction [shardingsphere-transaction]
      ✓ pom.xml
```



ShardingSphere 联邦查询引擎 Demo

SELECT * FROM t_order o INNER JOIN t_order_item i ON o.order_id = i.order_id WHERE o.order_id = 3;

以上面的 SQL 为例，t_order 和 t_order_item 存储在不同的数据库实例上，SQL Federation 会基于 RBO 进行谓词下推、列裁剪等优化，CBO 优化目前仍在开发中，目前只是基于 Calcite VolcanoPlanner 优化器实现逻辑关系代数到物理关系代数的转换。



ShardingSphere 联邦查询引擎 Demo

```
mysql> CREATE SHARDING TABLE RULE t_order_item(
```

```
-> RESOURCES(ds_0,ds_1),
```

```
-> SHARDING_COLUMN=user_id,
```

```
-> TYPE(NAME=HASH_MOD,PROPERTIES("sharding-count"=8)),
```

```
-> GENERATED_KEY(COLUMN=user_id,TYPE(NAME=snowflake,PROPERTIES("worker-id"=123)))
```

```
-> );
```

```
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> CREATE TABLE `t_order_item` (
```

```
-> `item_id` bigint(20) NOT NULL,
```

```
-> `order_id` bigint(20) NOT NULL,
```

```
-> `user_id` int(11) NOT NULL,
```

```
-> `status` varchar(50),
```

```
-> PRIMARY KEY (`item_id`)
```

```
-> ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
Query OK, 0 rows affected (0.18 sec)
```

```
mysql> INSERT INTO `t_order_item`(`item_id`, `order_id`, `user_id`, `status`) VALUES (1, 1, 1, 'TEST1'), (2, 1, 1, 'TEST2'), (3, 3, 2, 'TEST3'), (4, 3, 5, 'TEST4'), (5, 5, 3, 'TEST5');
```

```
Query OK, 5 rows affected (0.05 sec)
```

• t_order 和 t_order_item 分片数据存在跨库情况

```
mysql> SELECT * FROM t_order o INNER JOIN t_order_item i ON o.order_id = i.order_id WHERE o.order_id = 3;
```

order_id	user_id	status	item_id	order_id0	user_id0	status0
3	2	TEST3	3	3	2	TEST3
3	2	TEST3	4	3	5	TEST4

```
LogicalProject(order id=[0], user id=[1], status=[2], item_id=[3], order_id0=[4], user_id0=[5], status0=[6])
```

```
LogicalFilter(condition=[(CAST($0):BIGINT, 3)])
```

```
LogicalJoin(condition=[($0, $4)], joinType=[inner])
```

```
LogicalTableScan(table=[[sharding_db, t_order]])
```

```
LogicalTableScan(table=[[sharding_db, t_order_item]])
```

```
EnumerableHashJoin(condition=[($0, $4)], joinType=[inner])
```

```
EnumerableInterpreter
```

```
BindableTableScan(table=[[federate_jdbc, t_order]], filters=[(CAST($0):BIGINT, 3)])
```

```
EnumerableTableScan(table=[[federate_jdbc, t_order_item]])
```



01

Calcite 入门介绍

02

ShardingSphere 联邦查询引擎实现

03

ShardingSphere 联邦查询引擎规划



ShardingSphere 联邦查询引擎规划

- 统一 ShardingSphere 解析引擎和 Calcite 解析引擎，统一以 SqlNode 抽象语法树作为解析结果；
- 基于 ShardingSphere 系统库，收集 Calcite 优化所需的统计信息；
- 设计 ShardingSphere 分布式数据库代价模型， $cost = row_count + cpu + io + 网络传输$ ；
- 自定义部分物理算子，实现并发执行引擎，提供执行性能；
- 设计联邦查询引擎内存管理器，有效管控执行过程中的内存占用；
- 设计向量化执行引擎，进一步提升联邦查询引擎性能；

prestodb/presto Public

<> Code Issues 855 Pull requests 313 Discussions Actions Projects Wiki Security Insights

Improve cluster memory management #2624

Closed 4 of 5 tasks cberner opened this issue on Apr 1, 2015 · 8 comments

cberner commented on Apr 1, 2015 · edited

Contributor

Goals:

- Make it easier for users (query submitters) to understand resource limits. That is apply limits per-query, rather than per-task (users don't know what a task is)
- Prevent workers from running out of memory and crashing
- Better utilize the cluster's memory, rather than having a "big query" queue that limits ingest

Assumptions:

- The delta memory (including temporary allocations) required for an `Operator` to process a `Page` is linear, with a small constant, in the size of the `Page`
- Maximum number of running splits (for all tasks) per worker is relatively small (100s)
- Almost all queries only need a small amount of memory, but some need several orders of magnitude more
- We only need to track large memory allocations, everything else is insignificant

Other observations:

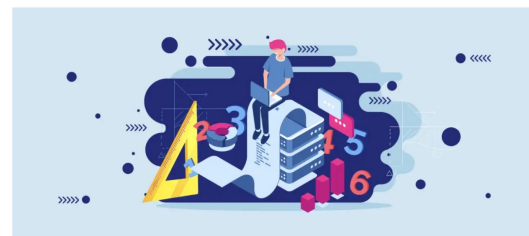
- It's infeasible to predict the memory usage of an arbitrary query in advance
- For many, but not all, Operators, it's easy to derive the maximum memory usage

Key Concepts:

PingCAP
Dec 2, 2019 · 12 min read · Listen

10x Performance Improvement for Expression Evaluation Made Possible by Vectorized Execution and the Community

Yuanjia Zhang



The query execution engine plays an important role in database system performance. TiDB, an open-source MySQL-compatible Hybrid Transactional/Analytical Processing (HTAP) database, implemented the widely-used Volcano model to evaluate queries. Unfortunately, when querying a large dataset, the Volcano model caused high interpretation overhead and low CPU cache hit rates.

谢谢观看

端正强

duanzhengqiang@apache.org



关注我们